GrantGuide Document Vault Implementation

Roadmap item 02 | Planned local release | October 25 2026

Build supporting documents into the application checklist. A researcher should be able to identify what a call requires, select the exact supporting files, verify their versions and relevance, and prepare a handoff for the sponsor's submission system. The document library supplies this workflow across applications within one project.

The product hypothesis is that connecting files to requirements and evidence reduces missing attachments and version mistakes. File storage alone adds little beyond an ordinary folder. Validate the workflow with applicants before expanding into a general document-management system.

1 Release scope and decisions

The first release runs in the existing local, single-user application and requires no paid services. Treat the baseline as the integration foundation; completion of baseline verification remains the responsibility of its implementation owner. This request produces an implementation specification, with application changes left for a subsequent implementation task.

Include in this release	Defer to later releases
PDF, TXT, Markdown and DOCX uploads with bounded local parsing	OCR, image intake, spreadsheets and arbitrary archives
A project document library, metadata filters and immutable file versions	Cross-project sharing, Drive synchronization and team permissions
Specific file versions selected for checklist requirements	Sponsor portal automation and submission integrations
Human-reviewed excerpts promoted into existing drafting evidence	Automatic scientific validation and automatic approval decisions
Manual ethics records with scope, phase and expiry	Institutional ethics-system integration
A saved handoff manifest, individual files and copyable existing draft answers	New sponsor narrative templates or DOCX-to-PDF conversion

Keep required attachment formats distinct from accepted vault formats. A DOCX can be stored and used as evidence even when the sponsor requires a PDF. The researcher supplies the final PDF, or an independently validated renderer from roadmap item 01 produces it.

Observable completion

One application has a required CV and a support letter. The researcher uploads and reviews both, links the correct versions, replaces the CV without losing its earlier version, resolves the resulting review notice, and creates a saved handoff. A separately recorded ethics approval is due at a later phase and remains visible without incorrectly blocking this submission. Restarting the application preserves all files, references and review decisions.

Product boundaries

Files and extracted text remain local. Uploading or linking a file does not send it to an AI provider. Existing explicit live-generation consent continues to apply only to selected, reviewed evidence. Use synthetic or redacted administrative documents for the first release; storage for participant-level study data needs a separate privacy design.

2 Research findings and their design consequences

The following sources establish workflow constraints. Numeric upload limits, storage structure and UI choices elsewhere in this specification are proposed application defaults, rather than sponsor rules.

Finding	Consequence for GrantGuide
NIH applications combine form fields with documents prepared externally and uploaded as PDFs. Some attachments have required format pages. S1	Represent portal answers and file attachments separately. Preserve researcher-supplied final files and explicit format instructions.
Wellcome's online workflow includes completing form fields and uploading supporting documents. S2	Make the final screen a preparation and handoff view, with copyable answers and downloadable files.
NIH allows IRB certification before award through Just-in-Time unless the funding office requires it earlier. Its February 18 2026 notice also distinguishes final approval from pending or expired dates. S3	Store the phase at which a record is required. Never apply one universal ethics gate to all initial applications.
OWASP recommends layered upload validation, private storage, generated storage names, size limits and authorized retrieval. S4	Validate before accepting bytes into the library; serve files only through application handlers.
Mammoth warns about untrusted input and disables external-file access by default. S5	Extract text only, keep external access disabled and treat parsing as untrusted processing.

Rules for sponsor instructions

Store the saved call instruction, its citation or visible manual override, the official URL and the date the researcher checked it. Where program-specific instructions differ from general guidance, show the discrepancy and require an explicit recorded choice. Do not derive a sponsor requirement merely from an example document or a generic checklist.

Every constraint must distinguish a checked rule from an unchecked one. A matching extension or a successful text extraction cannot establish that a file follows every sponsor layout rule. The handoff view displays the checks performed and the remaining manual review.

Validation of product value

Ask three to five applicants to assemble one realistic packet using their current process and the prototype. Record preparation time, missing or incorrect attachments, wrong versions, corrections and occasions when the tool requires duplicate entry. These are evaluation targets; no time-saving result has yet been measured. Prioritize fixes that improve this application workflow before adding more storage features.

3 User workflow and screens

Application checklist

Each document requirement gains an attachment area showing the selected file, version, review state and current issues. Actions are Upload, Choose from library, Review selected file, Replace selection and Open history. The picker is scoped to the application's project; changing an ID cannot bypass that boundary.

The requirement editor records the delivery method: file upload, portal text, external record or task. It also records when it is needed: application, pre-award, before research, a specific date, or unknown. Required or optional status and its source remain separate from delivery method and phase.

Uploading from a requirement opens a short form for title, category and owner. After storage and parsing, the researcher reviews the file and chooses whether it satisfies the requirement. A new upload never changes a requirement to Ready by itself.

Project document library

Add a Documents view within Project and evidence. Display title, category, owner, latest version, uploaded date, parse status, linked applications and archive state. Search is case-insensitive metadata search over title, filename, category, owner and tags. Full-document search, embeddings and cross-project search are outside this release.

Opening an item shows version history, original-file download, extracted text and warnings. Offer a plain text preview in the application. Opening an original PDF or DOCX uses an explicit download to the user's chosen viewer; do not render document HTML or activate embedded links and scripts.

Evidence selection

The researcher can select an exact extracted passage, review it against the original and promote it into a project evidence note. The UI displays the file version and extraction attempt behind that note. This action does not select the note for AI generation; the existing drafting selector remains explicit.

Preparation and handoff

Show three groups: files to upload, answers to enter and records needed at later phases. The researcher reviews missing files, superseded versions and unchecked format rules, then saves a handoff snapshot. Download actions use that snapshot. Display: "Prepared for transfer to the sponsor portal. Check the portal preview and submission receipt."

The existing Submitted stage remains a researcher-reported status. Downloading a file, copying an answer or creating a handoff never sets it automatically.

4 Current code and integration points

These observations describe the inspected working files on October 25 2026. There is no Git revision available in this checkout. Recheck the interfaces before implementing because baseline and roadmap item 01 work may change them.

Existing location	Current behavior	Required integration
`lib/schema.ts`	Schema version 1; source text up to 20,000 UTF-16 units; three fixed draft sections	Add vault records and migration; preserve source and section contracts until deliberately changed
`lib/repository.ts`	Serialized, atomic JSON state saves with expected-revision checks	Add coordinated metadata and immutable blob operations, recovery and idempotency
`components/project-panel.tsx`	Reads text and Markdown in the browser	Keep existing intake and add server file upload/library UI
`components/checklist.tsx`	Manual Ready status, source-note link and URL reference	Add pinned file selections, review actions, phase and delivery method
`lib/evidence.ts`	Quotes resolve to saved normalized source text and exact offsets	Bridge reviewed file excerpts to existing sources without changing citation meaning
`lib/readiness.ts`	Document status can satisfy an existing required item	Add file availability, review, phase and format checks
`lib/generation.ts`	Explicit selected sources; 50,000-character complete prompt bound	Admit only reviewed file-derived sources and preserve the existing prompt bound
`lib/exports.ts`	Markdown review packet with retained evidence	Add document references and handoff manifest; keep the existing review export
`app/api/workspace/route.ts`	Loopback request check; same-origin JSON mutations	Extract reusable guards and add a separate bounded binary route
`modules/documents/src/server.ts`	Local PDF/DOCX/text parsing, segments, SHA-256 of bytes and warnings	Wrap the existing ingestion API in bounded processing and validate its output

Important compatibility details

`normalizeText` uses a deterministic text hash, while server ingestion hashes the original bytes with SHA-256. Existing citation validation reconstructs the normalized text hash. Keep binary integrity hashes and citation hashes separate; passing the binary hash into an existing citation will break validation.

The ingestion module permits more types and more extracted text than this release. An application-level allowlist must run before it. Its DOCX directory-size check and asynchronous API do not provide an operating-system sandbox. The existing JSON body route's 120,000-byte limit is unsuitable for binary uploads.

The document vault can ship with the three starter sections. Complete sponsor narrative rendering belongs to item 01. Consume a renderer through an adapter if it becomes available; do not make file linking depend on it.

5 Data model and invariants

Use Zod at HTTP, repository, migration and parser-result boundaries. Add records to the next workspace schema version; choose the version number at integration time rather than colliding with another migration.

Record	Required fields and meaning
Document	`id`, `projectId`, title, category, owner label, tags, provenance, archived timestamp, ordered version IDs
DocumentVersion	`id`, `documentId`, monotonically increasing version, original display filename, detected MIME, byte length, SHA-256, server-only blob key, created timestamp, optional human-entered document date
ParseAttempt	`id`, `documentVersionId`, parser and version, status, warning codes, immutable normalized-text artifact key, extracted-text SHA-256, segments, start and finish timestamps
AttachmentSelection	`id`, `applicationId`, `requirementId`, `documentVersionId`, selected timestamp, review decision and reason, reviewer label, reviewed timestamp, requirement fingerprint
EthicsRecord	`id`, `projectId`, category, immutable revisions containing user-entered scope, institution, protocol/reference, status, dates, supporting version IDs and human confirmation
Requirement additions	Delivery method, phase, optional date, attachment constraints, ethics-record reference and provenance of those settings
HandoffSnapshot	`id`, application/project/call revisions, frozen requirement and review summary, selected version IDs and hashes, saved answers, filenames, warnings, created timestamp and rules version

Owner and reviewer labels are descriptive local metadata. They do not establish institutional authority or authenticated identity.

Invariants

1. A version's original bytes, filename, detected type and hash are immutable. Replacement creates a new version; it never overwrites a previous file.
2. All relationships resolve within the same project. Enforce this server-side for selection, evidence promotion, ethics links and downloads.
3. A selection pins a version. A newer document version triggers review but never silently changes the selected bytes. The researcher may keep the older version with a recorded reason.
4. Review is bound to both the file version and a fingerprint of the requirement's instructions, phase and constraints. Relevant requirement changes invalidate that review.
5. Parsing, attachment review and ethics confirmation are independent states. Successful parsing grants none of the other states.
6. Store file provenance explicitly. Real uploads are researcher-provided even when added while viewing a demo project; generated synthetic fixtures alone receive demo provenance.
7. Archive hides an item from default pickers; referenced versions remain readable. The first release has no selective permanent-delete operation.

Metadata bounds

Use current `Id` validation. Titles, owner labels and protocol labels are at most 500 characters; notes and scope descriptions at most 3,000; tags at most 20 entries of 80 characters. Reject invalid dates and broken references. Unknown dates are explicit null or unknown states, never guessed timestamps.

6 Upload storage and failure recovery

Proposed local limits

Resource	Release default
Input file	10,000,000 bytes; reject zero-byte uploads
Workspace physical blob storage	250,000,000 bytes including pending reservations
Inventory	100 documents per project; 20 versions per document
PDF extraction	At most 100 pages
Text extraction	At most 200,000 UTF-16 units per artifact; no silent truncation
Parsing	One worker at a time; 20-second hard deadline; at most four queued attempts
DOCX admission	At most 2,000 entries and 50,000,000 declared expanded bytes; reject malformed or inconsistent archives

These are product defaults. Capacity failures return recovery instructions; increasing a limit is a deliberate configuration change. Count retained old versions and extraction artifacts against storage; cap sidecar artifacts at a separate 50,000,000 bytes.

Storage layout and admission

Keep `workspace.json` as metadata. Put original bytes under `GRANTPATH_DATA_DIR/vault/blobs/<server-version-id>.bin`, parse artifacts under `vault/extractions/`, and temporary uploads under `vault/staging/`. Paths are generated from validated server IDs, resolved beneath the configured root, and never supplied by the client. Do not place files in `public/` or embed base64 in workspace JSON. Persist only artifact metadata in state; load extracted text through a bounded endpoint.

Validate the filename extension, detected content and declared MIME together. A missing or generic browser MIME can be accepted when content validation agrees; contradictory concrete types are rejected. Validate UTF-8 text, PDF signature and structure, and an actual DOCX package with required entries. Reject macro-bearing packages, external-file relationships, encrypted archives and unsupported embedded payloads. Do not extract archive paths to disk. Structural admission must precede library parsing.

Commit protocol

1. Check project access, request bounds, idempotency key and quota reservation. Stream at most the limit before buffering; clean up interrupted staging files.
2. Write a uniquely named staging file, compute SHA-256, validate admission and sync it. No document record exists yet.
3. Inside the repository queue, recheck expected revision, replacement target and quota. Atomically rename to a new immutable blob path, then atomically commit metadata plus the idempotency receipt.
4. Return the committed version as pending extraction. Parsing runs outside the repository queue. Persist its outcome through a separate internal serialized commit.
5. On restart, mark abandoned running parses retryable. Reconcile unreferenced staging/orphan blobs after a grace period; never delete a blob referenced by state or a handoff.

A crash between blob rename and metadata commit leaves an orphan, rather than metadata pointing to an unfinished file. Disk-full and conflict failures preserve existing versions. Reads verify existence, size and hash before serving or packaging; mismatches become visible storage issues.

7 Parsing and reviewed evidence

Use `ingestDocument` from the imported documents module. Run it in a disposable worker with a hard termination deadline and bounded input/output. A worker thread with a measured heap bound is sufficient for the supported local prototype; it is not a sandbox for hostile files. Hosted intake requires an isolated process/container with enforced memory, filesystem and network restrictions before release.

Parse states

State	User-facing behavior	Evidence eligibility
Pending or running	File retained; show progress and bounded retry	Ineligible
Parsed	Text and page/paragraph references available for review	Eligible only after an excerpt is reviewed
Partial	Show warning and pages/regions without text	Only explicitly reviewed extracted passages
No text	Explain that OCR or a text alternative is needed	Ineligible
Failed or limit exceeded	Original retained; show an actionable error	Ineligible

An unreadable or scanned file can still be a valid attachment. The researcher must inspect it externally and record readability and relevance before marking an attachment reviewed. A corrupt file that cannot be opened remains stored for recovery and cannot pass that review. Parsing success never proves sponsor formatting.

Do not enable OCR in the first release. A mixed scanned/text PDF must show incomplete extraction. A parser upgrade or retry creates a new ParseAttempt; it cannot modify previously cited text. Do not show raw parser stack traces, document contents or filenames in logs.

Bridge into existing sources

Promote a reviewed excerpt of at most 20,000 UTF-16 units through `normalizeText`, using a stable Source ID. Store a `fileOrigin` on that specific source version: document ID, file version ID, parse-attempt ID, full-artifact hash, and start/end offsets in the normalized extracted text. Verify exact text equality before saving. Preserve the origin through later draft snapshots.

The existing Citation keeps its source ID, source version, normalized text hash and offsets within the excerpt. The file origin supplies the route back to the full extraction and original bytes. Show PDF pages when available; DOCX paragraph references are not original page numbers.

Manual edits create a new source version with a manual-edit label and the earlier origin retained in history. Replacing a file marks dependent evidence for review; it never rewrites old source versions or quotations. Each file origin also records the document head seen at review. To keep an older excerpt after replacement, record a reason and create a new reviewed source version referencing the same original file and the newly reviewed head, even if its text hash is unchanged. A later replacement invalidates that review again. Keep the current limit of ten project evidence sources and ten selected draft sources; explain capacity failures without evicting notes.

Only reviewed promoted excerpts can enter generation. Preserve the existing 50,000-character total prompt limit; ask for a smaller selection instead of truncating. Ethics metadata and approval letters are excluded from automatic promotion and generation in this release.

8 Requirement readiness and manual ethics records

Readiness rules

Evaluate document checks in the context of a selected phase. Keep the current narrative and eligibility checks. Add the following deterministic rules with stable issue IDs and fingerprints that include selected versions, constraint revisions and applicable phase.

Condition	Result
Confirmed required file is needed now, with no selected version	Blocker for the current preparation phase
Selected blob is missing, corrupt, from another project or type-disallowed	Hard failure; cannot be waived into a downloadable file
File review is absent, or instructions changed after review	Review blocker
A newer version exists	Review blocker until the researcher selects it or explicitly retains the older version with a reason
Required PDF has only a selected DOCX	Format blocker; request a final PDF
Required rule is not machine-checkable	Manual review item; never label it automatically passed
Required phase or delivery method is unknown	Resolve the ambiguity before marking that required item prepared
Record is needed at a later confirmed phase	Show a future task; do not block the current phase

The existing Ready status remains a user statement. For file-upload requirements it must be accompanied by an available reviewed selection. An external URL can support an external-record requirement but cannot satisfy a required uploaded file. Unknown, missing and failed states remain distinct.

Existing waiver behavior must not conceal unavailable bytes, cross-project access or failed integrity checks. Ordinary requirement waivers remain visible in the handoff and cannot create a green machine-verified result. A preparation snapshot with unresolved items is allowed as an explicitly incomplete review handoff.

Ethics records

Support IRB, animal-study, data-use, training, conflict-of-interest and other categories. A researcher enters the institution, protocol/reference, actual scope, status, approval date, expiry meaning and optional supporting file versions. Statuses include unknown, pending, approved, exempt determination, not applicable, expired and withdrawn. Exempt and not applicable require a human-entered basis and reference; AI cannot assign them.

Allow expiry to be a confirmed date, explicitly no expiry stated, or unknown. Do not infer an annual renewal. A recorded expiry is inclusive through that date; show overdue on the following workspace date. Use the existing `America/New_York` display-date basis, show it, and preserve source wording. Compare expiry with the relevant required date where known; an approval expiring before planned research starts needs review even if it is current today.

Each confirmation pins the supporting versions and scope revision. New files, scope edits, status changes or a different required phase invalidate affected confirmations. An expired record generates a scoped review task; it never invents a prohibition on the whole project. Allow one active generated task per record and task kind; supersede older generated tasks when its revision changes, retaining history. Use in-app agenda and existing calendar exports only; scheduled messages remain roadmap item 07.

9 HTTP and mutation contracts

Extract the existing local request checks into a shared server helper that distinguishes JSON and binary routes. Bind the server to loopback. Require exact same-origin Origin for mutations, reject cross-site Fetch Metadata where present, expose no cross-origin access and keep responses `no-store`. Local access is a single trusted operating-system user boundary, not institutional authentication.

Proposed endpoints

Endpoint	Contract
`POST /api/documents`	Raw `application/octet-stream` body with bounded, base64url-encoded JSON metadata in `X-Upload-Metadata`; header at most 4,096 bytes. Metadata contains project ID, optional replacement document ID, display filename, claimed MIME, title, category, owner label, expected revision and idempotency key. Server detects the actual type.
`POST /api/documents/{id}/versions/{versionId}/parse`	Same-origin JSON: expected revision and idempotency key. Queues an explicit retry; rejects duplicate active attempts.
`GET /api/documents/{id}/versions/{versionId}`	Bounded version and parse metadata for the resolved project. No internal paths or stack traces.
`GET /api/documents/{id}/versions/{versionId}/text?attemptId=...`	Bounded normalized text and segments; original file access is checked first.
`GET /api/documents/{id}/versions/{versionId}/download`	Verified immutable bytes; attachment disposition, sanitized filename, detected MIME, `nosniff`, no inline HTML.
`POST /api/applications/{id}/handoffs`	JSON: expected revision, phase and idempotency key. Freeze selections, answers, checks and warnings.
`GET /api/applications/{id}/handoffs/{snapshotId}`	Saved manifest and snapshot-specific download targets. A manifest is not a bearer credential.

Use the existing `/api/workspace` mutation envelope for metadata commands: editDocumentMetadata, archiveDocument, selectAttachment, reviewAttachment, promoteFileExcerpt, saveEthicsRecord and confirmEthicsRecord. Include explicit target IDs, expected state revision and bounded payloads. Derive blob paths and relationships on the server. Browser progress may use upload bytes; parse status is read from saved metadata.

Concurrency and errors

State revision protects researcher mutations. Internal parse-result commits reload the latest state and change only their matching attempt; they do not overwrite user edits. Preserve client buffers on 409 and display the new revision. Background updates can cause a 409; offer a refresh and review, without replaying a destructive edit.

Persist upload and handoff receipts for seven days. Bind each key to the logical payload fingerprint and body SHA-256. A replay with identical input returns the original resource before revision checks; a reused key with different input returns 409. Recheck versions and integrity even on a download after replay.

Return 400 for invalid metadata, 403 for origin/access failure, 404 for unavailable records, 409 for stale state or conflicting keys, 413 for byte/text limits, 415 for unsupported types, 422 for rejected structure, 429 for a full parse queue and 503 for storage/quota unavailability. A post-admission parse failure updates the retained version; it does not turn a successful stored upload into a lost file.

10 Handoff output and sponsor portal transfer

Saved snapshot

Store the selected application revision, project revision, call/source versions, preparation phase and rules version. Freeze requirement instructions, constraint checks, file selections and review decisions. Include the relevant ethics-record revisions and later-phase tasks as internal notes. Copy the existing draft answer text into the snapshot, with the revision and evidence references used.

For each selected file, record requirement ID, document/version IDs, SHA-256, original filename, transfer filename, MIME, byte length and readiness state. The saved snapshot always refers to the selected bytes, even after a new upload. Later changes mark the snapshot as older than current work without rewriting it.

Outputs in the first release

1. A readable preparation manifest with files, portal-field mappings, missing items and future tasks, plus a JSON manifest for traceability.
2. Individual downloads of selected files. Original bytes are preserved; do not flatten, merge, re-sign or convert uploads automatically.
3. Copyable plain text for explicitly mapped existing draft sections. Show the target portal field label and confirmed limit, if known. Arbitrary new sponsor section templates depend on item 01.
4. The existing Markdown review packet, extended with file/version references and internal evidence notes.

A ZIP convenience download can follow after this workflow works; it is not required. The manifest and review packet are internal preparation documents and must be visually separated from sponsor attachments. Do not inject readiness warnings, provenance appendices or internal ethics notes into an uploaded sponsor narrative.

Filenames and format checks

Use safe unique transfer names, such as `cv_a1b2c3d4_v3.pdf`, while preserving original filenames in the manifest. Default to ASCII letters, digits, underscores and hyphens with an extension and a 50-character total cap; resolve collisions deterministically. NIH specifically requires unique descriptive filenames of at most 50 characters. S1 Other program requirements are saved as explicit application constraints.

Check MIME and byte-size rules deterministically. Check PDF page count when available. Treat font, margin, table fidelity, signatures, flattened form behavior and content suitability as manual checks unless a validated checker exists. A parse failure does not silently remove a selected file from the manifest. Invalid or missing bytes disable that file's download and produce a visible incomplete state.

Completion in the portal

Provide the official portal URL as an ordinary user-opened link; do not fetch it or attach credentials. The researcher or institution transfers files, enters answers, reviews the assembled portal preview and submits through the required process. NIH advises checking its application preview and post-submission rendering. S1 Any receipt/reference entered in GrantGuide is labeled researcher-reported.

11 Migration security and retention

Migration from the existing workspace

Read the old schema explicitly, validate it, and create an exclusive, timestamped backup before the first write. Convert through a pure migration function, validate the complete new state and atomically save it. Repeated startup must not duplicate records. If the backup or migration fails, preserve the original and return an actionable storage error; never initialize a replacement demo workspace.

Add empty document, ethics, handoff and operation-receipt collections. Preserve existing source text, hashes, IDs, requirement history, draft snapshots and labels. Keep section keys and the three-section constraint until item 01 deliberately changes them. Retain existing source links and URLs as legacy references. A previously Ready document requirement needs an explicit delivery-method review before it can be treated as a file selected for handoff; migration must not invent a file from a URL.

Requirements with no phase are labeled unknown until reviewed. Preserve their previous status and display the resulting new review issue rather than silently changing the historic record. Do not infer that old IRB tasks are due at initial submission.

Update demo reset to protect every retained reference, including handoffs and researcher uploads linked to a demo project. Preserve any referenced demo parent or remap it through an explicit tested operation. A reset may clear synthetic records only when no retained researcher data depends on them. Never delete blobs merely because their parent has demo provenance.

Local release controls

Check all project and parent-child relationships server-side. Keep storage paths private, generate them from IDs, prevent path traversal and verify resolved paths remain under the vault root. Downloads require the same local request boundary as metadata. Block cross-site GETs where browser Fetch Metadata is available; do not expose CORS reads. Host-header checks alone are not hosted authentication.

Parse with fixed library options and no network or filesystem references from document content. Render extracted text as escaped text nodes. Treat any instruction inside a file as untrusted source material. Do not log uploaded bytes, extracted passages, protocol identifiers or provider secrets. Error codes and operation IDs suffice for diagnosis.

Hosted release prerequisites

Before any hosted or shared use, add authenticated membership and object-level authorization, private storage, enforced parser isolation, upload abuse controls, a malware/quarantine policy, encryption/key management, backup/restore verification and a deletion/retention design. Configure operating-system access controls explicitly; POSIX file modes alone do not establish Windows ACL protection. Hosted deployments must fail closed if these facilities are absent.

Retain files until the local workspace is explicitly removed. Archive is not deletion; show that retention rule at upload. Back up metadata and blobs under the same repository lock so the backup is internally consistent. The local prototype performs no third-party malware upload or cloud scanning, and must not claim that a parsed file has been certified safe.

12 Implementation sequence and ownership

The requesting user is product owner. A single implementation lead owns integration and shared contracts. No additional human approvers or worker accounts are assumed. This document's author owns only its source, PDF and local rendering helpers; implementation file ownership must be claimed before code edits begin.

Work package	File set and dependency	Completion evidence
V02 A Contracts and migration	`lib/schema.ts`, new migration module and fixtures; first	Idempotent old-state migration and reference validation; original backup preserved
V02 B Blob storage and routes	New `lib/vault/` storage module, shared route guard, document routes and repository integration; after A	Bounded uploads, restart recovery, integrity checks and retry receipts
V02 C Extraction and evidence	New parsing worker/wrapper, `lib/evidence.ts`, `lib/generation.ts`; after B	PDF/DOCX/text parsing, failure retention and exact file-to-excerpt traceability
V02 D Checklist and library	New document components, project/checklist components and readiness; after A and B	Choose, replace, review and pin versions through one persisted application flow
V02 E Ethics and agenda	Ethics components, readiness and calendar adapters; after D	Phase-aware scope/expiry behavior and deduplicated agenda tasks
V02 F Handoff	New handoff module/routes/UI and `lib/exports.ts`; after C through E	Immutable manifest, exact original file downloads and copyable answers
V02 G Verification and handoff	Focused domain/route/browser tests and project documentation	Prior workflow plus new acceptance matrix pass on the same final files

First implementation slice

Start with one required CV: migrate a fixture workspace, upload one PDF, retain its bytes, select it for a requirement, review it, restart and download the same hash. Establish storage integrity before expanding parsing, ethics and export UI. Do not start with a standalone file-manager screen.

Shared file stewardship

The implementation lead and requesting user are the steward and human owner for contracts and migrations. Package manifests and lockfile changes require that same exclusive ownership. Avoid `app/layout.tsx` changes; keep new styles scoped where practical. If global styles must change, claim them explicitly as part of the UI package. There is no concurrent writer authorized by this plan.

Reuse the imported documents API. Prefer new application adapters over modifying canonical modules. If a shared parser defect requires a module change, read its README and scoped instructions, run its module checks and record the upstream customization. New dependencies require a concrete reason and lockfile update; neither ZIP export nor OCR is a prerequisite.

Implementation handoff

Record actual changed files, migration version, configuration names, limits, passing commands, browser evidence and remaining unverified capabilities. Do not claim hosted protection, submission success or institutional approval from local checks. Keep this specification and acceptance evidence aligned with the final implementation.

13 Acceptance and verification plan

Use synthetic fixtures and temporary data directories. Ordinary tests make no external API calls. Test failures must preserve previously saved application work.

Scenario	Required result
Initial application flow	Upload, parse, review, select, restart and download a synthetic PDF; file SHA-256 is unchanged
Replacement and concurrency	v1 remains referenced after v2; no silent relink; two simultaneous replacements cannot share a version number; stale edits return 409
Interrupted upload and commit	Disconnect, timeout, disk-full and injected crash points leave no false-ready metadata; orphan recovery preserves referenced files
Idempotency	Same key and bytes return the original resource after a lost response; different bytes with that key return 409
Admission limits	Oversized chunked body, false length, spoofed type, path-like filename, macro package, malformed ZIP and expansion bomb are rejected without overwriting state
Parsing failures	Scanned, encrypted, partial, malformed and over-limit files retain their admitted originals; unsupported evidence is excluded; the worker terminates on timeout
Unicode and provenance	BOM, CRLF and emoji passages retain exact normalized offsets; binary hashes and citation hashes remain distinct; real uploads keep researcher labels
Evidence updates	Source quotes resolve to exact file version/attempt; replacement marks dependent evidence stale; edited text is labeled manual; missing origin never passes validation
Requirement checks	Required current-phase file needs an available reviewed selection; a required PDF is not satisfied by DOCX or a URL; changed instructions invalidate review
Ethics timing	Later-phase IRB record does not block initial application; unknown phase prompts review; expired, no-expiry and unknown dates remain distinct; scope edits invalidate confirmation
Handoff stability	Snapshot selections, answers and warnings stay unchanged after edits; each download matches its saved hash; missing bytes are visible; internal notes stay out of sponsor files
Access and reset	Cross-project IDs, cross-site mutations and arbitrary paths fail; reset cannot orphan researcher uploads or handoffs; metadata and blob backups restore together
Compatibility	Existing text intake, call review, drafts, readiness and calendar exports still work; fixture/live labels and usage consent survive migration

14 Verification commands and manual release journey

Run these checks against the final integrated implementation. Reinspect package scripts first because baseline work may change them.

Command	Purpose
`npm ci`	Install exactly the dependencies in the implementation's lockfile
`npm run check`	Run the repository's type, test and production-build checks
`npx vitest run modules/documents/tests`	Verify the reused ingestion module when adapters or parser code change
`node --import tsx modules/documents/examples/demo.ts`	Exercise the local ingestion example without provider credentials
`npm run dev`	Start the local app for the persisted browser journey

Use the repository's browser-test command if its script exists; otherwise add a bounded synthetic journey under its existing test conventions. All test and manual verification data belongs in an isolated temporary `GRANTPATH_DATA_DIR`, never the researcher's normal workspace.

Measure parse duration and peak memory on fixture PDF/DOCX/text files near the release limits, and verify the app remains responsive while parsing. The 20-second deadline and quotas are enforced limits, rather than claims about achieved performance.

Manual release journey

1. Create a researcher-labeled synthetic project and a saved call. Add two current-phase attachment requirements and one later-phase ethics record with confirmed scope and timing.
2. Upload, review and select a CV and a support letter. Confirm that neither upload automatically satisfies its requirement.
3. Replace the CV, inspect both versions and resolve the resulting review issue. Verify that the earlier selection remains unchanged until explicitly replaced.
4. Promote an exact extracted passage, save a draft using it and follow its citation back to the original file version.
5. Create a handoff, download and open every selected file, and compare each hash with the manifest. Confirm that internal notes are separate from sponsor attachments.
6. Restart the application and inspect saved reviews, history and handoff downloads. Repeat with an unreadable file and a changed call instruction.

Record the implementation's file revision or available Git SHA, exact commands and exit codes, browser evidence, fixture names and data-directory configuration. List any skipped prerequisite and the capability it leaves unverified. A failed storage or access check prevents release of that path; it cannot be replaced by a successful UI screenshot.

15 References and implementation entry point

Sources were inspected on October 25 2026. Recheck the selected call and current sponsor instructions when implementing a sponsor-specific rule. The documents below explain the general workflow and engineering constraints; they do not establish support for every sponsor or portal.

- **S1 NIH Format Attachments.** Externally prepared PDF attachments, filename rules, format pages and preview checks. Official guidance
- **S2 Wellcome application guidance.** Online form completion and supporting uploads. Official guidance
- **S3 NIH NOT-OD-26-043.** Released February 18 2026; timing and status of IRB certification. Official notice
- **S4 OWASP File Upload Cheat Sheet.** Layered admission, storage and retrieval controls. Primary guidance
- **S5 Mammoth documentation.** Untrusted document processing and external-file access. Maintainer documentation

Local sources

- `C:/Users/vzhu0/Downloads/GrantPath_Future_Capabilities_Roadmap.pdf`, item 02 and shared release gate.
- `docs/PROJECT_BRIEF.md`, `docs/OWNERSHIP.md`, `docs/API_CONTRACT.md`, `docs/QUICKSTART.md` and root `AGENTS.md` for accepted project boundaries.
- `lib/schema.ts`, `lib/repository.ts`, `lib/evidence.ts`, `lib/generation.ts`, `lib/readiness.ts`, `lib/exports.ts` and `lib/dates.ts` for current domain behavior.
- `components/project-panel.tsx`, `components/checklist.tsx` and `app/api/workspace/route.ts` for current intake and request boundaries.
- `modules/documents/README.md`, `modules/documents/src/server.ts`, `modules/documents/src/index.ts` and `packages/runtime/src/body.ts` for reusable normalization, parsing and request limits.

Decisions ready for the next task

Proceed with the local release defaults in this specification. No sponsor account, provider key or institution API is needed to build its synthetic end-to-end workflow. The first implementation task is V02 A plus the smallest V02 B slice for one PDF-backed requirement. Keep the exact sponsor's attachment instructions as researcher-confirmed input; a generic example must not become an official requirement.

The next coding session should reread this specification, claim the affected file set, inspect the current schema and build status, and implement that persisted upload-to-requirement slice. Extend to reviewed evidence, ethics and handoff only after the storage and versioning checks pass.